ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

NT106 — LẬP TRÌNH MẠNG CĂN BẢN

CẨM NANG LABGUARD

HƯỚNG DẪN THÀNH VIÊN D

KIỂM THỬ TCP & HỖ TRỢ XỬ LÝ SỰ CỐ

Đỗ Minh Ngọc: hỗ trợ xử lý sự cố dựa trên bằng chứng trong LabGuard Admin

---

BIÊN SOẠN: VÕ TRỌNG PHÚC

NHÓM THỰC HIỆN: 4 THÀNH VIÊN

TP. HỒ CHÍ MINH — 2026

KHỞI ĐỘNG NHANH DÀNH CHO THÀNH VIÊN D

(Đọc xong trang này trong 2 phút là bạn biết chính xác mình cần làm gì hôm nay)

1. BẠN LÀ AI?

Bạn là **Đỗ Minh Ngọc**, phụ trách **Công cụ kiểm thử truyền thông TCP và chức năng hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard**. Phần hỗ trợ nằm trong Admin; Gateway/Collector chỉ là tùy chọn.

2. BẠN LÀM GÌ?

- Hiểu giao thức Protocol do Thành viên A thiết kế.
- Dựng môi trường Stub/Mock (FakeAgent và FakeCentral) để thử nghiệm độc lập.
- Nắm vững cơ chế hàng đợi có giới hạn để điều tiết lưu lượng gói tin.
- Thiết kế logic gợi ý có bằng chứng trong chi tiết sự cố, phối hợp Quang/Vy.
- Kiểm tra thiếu/cũ/lỗi nguồn; không tự kết luận nguyên nhân hoặc tự sửa máy.

3. BẠN KHÔNG CẦN LO PHẦN NÀO?

- Bạn **không cần** lo đo phần cứng máy trạm (việc của **Thành viên A**).
- Bạn **không cần** lo cấu trúc bảng CSDL SQL Server hay viết luật nghiệp vụ (việc của **Thành viên B**).
- Bạn **không cần** lo vẽ giao diện đồ họa WinForms (việc của **Thành viên C**).
- *Lưu ý:* Simulator và Wireshark là **công cụ hỗ trợ kiểm thử sau này**, không phải nghiệp vụ cốt lõi bạn cần lo ngay từ đầu.

4. HÔM NAY BẮT ĐẦU Ở ĐÂU?

Đọc phân công và WBS D-007..D-010, đối chiếu dữ liệu thiết bị/sự cố bắt buộc với B/C; giữ nhiệm vụ TCP Echo/Relay và giao thức đã đóng băng.

5. FILES BẠN CẦN MỞ TRƯỚC:

1. `docs/internal/TEAM_ASSIGNMENTS.md`: Phân công chính thức và ranh giới tích hợp.
2. `src/LabGuard.Protocol/FrameCodec.cs`: Cách cắt và ghép 4 byte độ dài khi trung chuyển dữ liệu.
3. `src/LabGuard.Protocol/WireEnvelope.cs`: Cấu trúc thông điệp chuẩn để kiểm tra tính toàn vẹn.
4. `src/LabGuard.Protocol/MessageTypes.cs`: Danh mục tên các loại thông điệp cần kiểm thử.

MỤC LỤC

Hiện trạng và phân công sau PR #11 **1**

1 Kiểm thử TCP và hỗ trợ xử lý sự cố **2**
1.1 D-007 đến D-010: từ bằng chứng đến gợi ý có giới hạn 2
1.2 Phối hợp Quang và Vy 2
1.3 Ví dụ kế hoạch MAY-03 — dữ liệu hoàn toàn minh họa 3
1.4 Sản phẩm bàn giao và kiểm tra dự kiến 3

2 Tham khảo tùy chọn: Gateway, Collector và kiểm tra tải **3**
2.1 Bài toán thực tế: Khi phòng máy có hàng trăm máy tính 3

3 Các bài học TCP và mở rộng tùy chọn **5**
3.1 Chặng 1: CÔNG CỤ ĐÃ CÓ PR #9: Môi trường Stub độc lập (FakeAgent / FakeCentral) 5
3.2 Chặng 2: THAM KHẢO TÙY CHỌN: Hàng đợi thu gom an toàn (Bounded Channel) 5
3.3 Chặng 3: THAM KHẢO TÙY CHỌN: Chuyển tiếp hai chiều (Duplex Relay) & Tự phục hồi (Failover) 6
3.4 Công cụ hỗ trợ: Bộ giả lập phòng máy (Simulator) 7
3.5 Công cụ hỗ trợ: Bắt và phân tích gói tin bằng Wireshark 8
3.6 Kịch bản kiểm thử dự kiến cho nhiệm vụ chính của Ngọc 9

4 Năm bẫy lỗi lập trình Gateway & Kiểm thử mạng **10**

5 Câu hỏi tham khảo TCP và mở rộng tùy chọn (20 câu) **11**

Hiện trạng và phân công sau PR #11

TÓM TẮT 30 GIÂY

Mốc nguồn: main `70481fa`, PR #11 đã merge. Đây là cẩm nang học và kế hoạch hiện thực; các mô tả tương lai/pseudocode không phải bằng chứng tính năng đã hoạt động.

- **Agent đã có:** chương trình Console, cấu hình JSON, DeviceId riêng cho mỗi bản cài đặt, đo CPU/RAM/thời gian hoạt động Windows tại máy, chế độ thử TCP/HELLO, hủy tác vụ và ghi nhật ký.
- **Chế độ giám sát bị chặn có chủ đích** tới khi TLS/phiên kết nối/AUTH sẵn sàng. MetricsService/HeartbeatService đã có thành phần và kiểm thử nhưng chưa được bật trong chương trình khởi chạy.
- **Chưa hoàn tất:** A-005 kết nối lại, A-006 ReplayBuffer, đọc phản hồi/kiểm tra mất kết nối, tích hợp SQL/ACK và Windows Service. B-001 đã có bộ tiếp nhận TCP; B-005 mới có DDL, còn đang làm. C-001 có bố cục 0.1, chưa nghiệm thu liên kết dữ liệu và quy trình xử lý.
- **Kiểm thử đã báo cáo từ PR #11:** 195 bài đạt trên Windows tại máy: Protocol 107, Agent 64, Central 9, MockHarness 15. Đây **không phải kết quả GitHub CI**. Số 34/165 trong hồ sơ cũ là mốc lịch sử.

Thành viên	Trách nhiệm chính thức
A — Võ Trọng Phúc	Agent và Protocol.
B — Nguyễn Bá Quang	Central Server và SQL Server.
C — Lý Tuyết Vy	Admin Windows Forms.
D — Đỗ Minh Ngọc	Công cụ kiểm thử truyền thông TCP và chức năng hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard.

Nguồn: quyết định trực tiếp của người điều phối dự án ngày 17/10/2026; không tự tạo biên bản họp hoặc chữ ký. Chức năng hỗ trợ xử lý sự cố là **phần việc chính thức trong kế hoạch**, chưa phải chức năng đã được lập trình. Logic nhúng trong quy trình xử lý sự cố của LabGuard Admin, dùng dữ liệu có thẩm quyền qua Central; D phối hợp B/C, không làm ứng dụng TCP độc lập.

Gateway/Collector và Simulator là **mở rộng P1 tùy chọn** sau khi phần cốt lõi ổn định. Nội dung mở rộng được giữ để học, không phải cam kết chính của Ngọc. GK là một buổi trình bày đề xuất theo lịch giảng viên, không phải hạn hoàn thành đồ án; các chặng là kế hoạch tương đối toàn dự án.

B-009 cần đối chiếu kế hoạch: WBS cũ nhắc AssignedToUserId/IncidentHistory nhưng schema hiện chỉ có AssigneeId và chưa có bảng lịch sử. Giữ bản ghi CHƯA LÀM để Quang/nhóm quyết định, không tự sửa schema hoặc coi bảng/cột cũ đã tồn tại.

Năm mốc khác nhau: kết nối TCP; xác thực ứng dụng; nhận số liệu; ghi thành công SQL; nhận ACK ứng dụng. Chế độ thử trả exit 0 chỉ cho biết lệnh kết thúc; phía bên kia đóng kết nối có thể chưa được phát hiện. Nhật ký kết nối không chứng minh đã xác thực hoặc lưu SQL.

Bắt đầu đúng cách: đọc `docs/agent_deployment/README.md` và `docs/internal/TEAM_ASSIGNMENTS.md`. WBS giữ 34 nhiệm vụ cũ và sáu nhiệm vụ bổ sung cho chức năng hỗ trợ xử lý sự cố, đều chưa làm. Nâng cấp .NET 10 là việc riêng toàn bộ dự án, chưa thực hiện trong lượt đồng bộ này.

1. Đỗ Minh Ngọc: kiểm thử TCP và hỗ trợ xử lý sự cố

TÓM TẮT 30 GIÂY

D-002 TCP Echo đã XONG qua PR #8; D-001/D-003 chưa có nghiệm thu. Bộ kiểm thử mô phỏng của PR #9 đã có, không thay D-003. D-007..D-010 là nhiệm vụ chính thức theo quyết định người điều phối 17/10/2026, **đều CHƯA LÀM**. Chức năng hỗ trợ phải nằm trong Admin, không thay bằng chương trình chẩn đoán TCP độc lập.

1.1 D-007 đến D-010: từ bằng chứng đến gợi ý có giới hạn

1. **D-007 — rà nguồn:** dữ liệu bắt buộc/tùy chọn, quyền truy cập, độ mới, phần thiếu/cũ và ý nghĩa từng trường. Không suy tình trạng SQL hay thời điểm đo khi chưa có nguồn xác nhận.
2. **D-008 — thiết kế quy tắc:** mỗi nhận định có bằng chứng, giới hạn và hướng kiểm tra. Bao phủ CPU, RAM, mất liên lạc, dịch vụ dừng; phân biệt điều kiện đã phục hồi với trạng thái xử lý sự cố.
3. **D-009 — logic nội bộ:** thư viện dùng lại do Admin gọi, nhận dữ liệu B/C đã thống nhất; không truy vấn SQL/Agent trực tiếp, không tự thay đổi dữ liệu hoặc đóng sự cố.
4. **D-010 — xác minh tích hợp:** sau D-009, B-010 và C-012, chạy sáu ca với nguồn và luồng xử lý thật. Phân biệt thiết kế, mã đã viết và kiểm thử đã chạy; mô phỏng riêng không chứng minh toàn hệ thống hoạt động.

1.2 Phối hợp Quang và Vy

Quang (B-010) cung cấp dữ liệu/lệnh có thẩm quyền qua Central; Vy (C-012) phụ trách giao diện, quy trình xử lý, quyền và kiểm tra xung đột cập nhật. Hai giờ B và ba giờ C chỉ là phần ghép khi chức năng cốt lõi đã sẵn sàng; xác thực, lưu dữ liệu và kết nối còn thiếu phải tính riêng. Ngọc giữ công cụ kiểm thử TCP và logic gợi ý, không nhận thay phần cốt lõi của A/B/C.

Giai đoạn đầu dùng dữ liệu thiết bị/sự cố được phép lấy qua Central và hiện rõ bằng chứng còn thiếu. Giai đoạn sau chỉ khi nhóm chấp thuận và phần xử lý đã sẵn sàng mới lấy lịch sử SQL qua thiết kế tùy chọn `GET_METRICS` / `METRICS_RESULT`. Không tạo thời điểm, lịch sử hoặc tình trạng SQL giả; không sửa schema/giao thức đóng băng.

1.3 Ví dụ kế hoạch MAY-03 — dữ liệu hoàn toàn minh họa

KHÁI NIỆM MẠNG CỐT LÕI

Bằng chứng: giả định MAY-03 đang kết nối (ONLINE), ba mẫu CPU minh họa 91.2% → 93.1% → 92.4%, RAM 58.5%, sự cố HIGH_CPU đang mở (OPEN). Đây không phải số đo hoặc sự cố từ phần mềm đã chạy.
Nhận định: ba mẫu vượt ngưỡng CPU 90% của thiết kế phù hợp một cảnh báo HIGH_CPU; không xác định chắc tiến trình/nguyên nhân. ONLINE là kết nối, HIGH_CPU là cảnh báo, OPEN là trạng thái xử lý: không gộp thành một trạng thái chung.
Hướng kiểm tra: kiểm tra độ mới/nguồn của mẫu, trao đổi tác vụ đang chạy, đối chiếu tiến trình trên máy bằng người có quyền. Nếu thiếu mẫu hoặc chỉ có dữ liệu cũ, hiện rõ thiếu bằng chứng thay vì khẳng định quá tải hiện tại.
Thao tác xử lý: người quản trị xác nhận, bắt đầu hoặc hoàn tất xử lý theo quyền, qua bộ xử lý lệnh Central khi đã hiện thực. Chức năng hỗ trợ không tự đóng sự cố hoặc chạy lệnh sửa từ xa. Điều kiện phục hồi không tự chuyển trạng thái xử lý sang RESOLVED.

1.4 Sản phẩm bàn giao và kiểm tra dự kiến

Bảng ánh xạ dữ liệu, quy tắc giải thích, thư viện dùng trong Admin, vùng hiển thị cùng C và hồ sơ sáu ca sau tích hợp. Các ca phải kiểm tra nguồn, phần thiếu/cũ, trạng thái, quyền và phản hồi xung đột. Chưa kết luận đạt khi chưa chạy mã với nguồn thật. DeviceId thuộc Agent; chức năng hỗ trợ dùng thiết bị đang chọn, không yêu cầu nhập lại định danh.

2. Tham khảo tùy chọn: Gateway, Collector và kiểm tra tải

TÓM TẮT 30 GIÂY

Phần tiếp theo giữ kiến thức mở rộng của cẩm nang cũ. Gateway/Collector/Simulator thuộc P1 sau khi phần cốt lõi ổn định, **không phải cam kết chính của Ngọc**. Các diagram, pseudocode và kịch bản mở rộng là đề xuất học tập, không phải hiện trạng triển khai.

2.1 Bài toán thực tế: Khi phòng máy có hàng trăm máy tính

Khi hệ thống LabGuard được nhân rộng cho cả một dãy phòng máy của trường đại học với hơn 500 đến 1,000 máy tính hoạt động cùng lúc:

- **Nguy cơ bão kết nối (Connection Storm):** Khả năng quá tải phụ thuộc lưu lượng, I/O và tài nguyên thực tế; chỉ xem xét scale sau khi đo đúng workload, không giả định số lượng socket tự gây nghẽn.
- **Mạng giữa các phòng học (WAN / NAT):** Các máy trong từng phòng thường nằm sau router riêng, Collector L7 là hướng gom dữ liệu có điều kiện; hiệu quả cần đo thực tế. Gateway L4 và Collector L7 không phải cùng một thành phần.

KHÁI NIỆM MẠNG CỐT LÕI

Nội dung tham khảo mở rộng, không phải nhiệm vụ chính:

1. **Collector L7 tùy chọn:** Gom các bản tin số liệu từ các máy trong phòng, đưa vào hàng đợi an toàn và chuyển tiếp về máy chủ trung tâm.
2. **Môi trường giả lập độc lập (FakeAgent / FakeCentral):** Giúp nhóm kiểm thử sớm mà không phải đợi nhau viết xong code.
3. **Bộ giả lập phòng máy (Simulator):** Hướng mở rộng tùy chọn để mô phỏng nhiều máy trạm và tình huống lỗi. Hiện chỉ có bản khung; chưa có bằng chứng sinh tải hoặc hiển thị số liệu/cảnh báo trên Admin.
4. **Phân tích gói tin qua Wireshark:** Soi dòng byte nhị phân để chứng minh hệ thống đóng khung 4 byte Big-Endian chuẩn xác.

Hình này chỉ cần nhìn theo các khối chức năng từ trên xuống dưới theo thứ tự triển khai.



Figure 1: Sơ đồ tham khảo mở rộng tùy chọn, không phải roadmap chính của Ngọc

Ý chính: Ngọc ưu tiên Công cụ kiểm thử truyền thông TCP và chức năng hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard; sơ đồ mở rộng trên chỉ là tài liệu P1 tùy chọn, chưa có kết quả tải hay failover.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Gateway là trạm trung gian kết nối giữa máy phòng học và máy chủ trung tâm.
- Simulator và Wireshark là công cụ hỗ trợ để thẩm định độ ổn định và kiểm chứng hệ thống

dưới tải lớn.

3. Các bài học TCP và mở rộng tùy chọn

TÓM TẮT 30 GIÂY

- Chặng 1: Xây dựng FakeAgent và FakeCentral để kiểm thử độc lập.
- Chặng 2–3: Lập trình hàng đợi Bounded Channel và chuyển tiếp hai chiều.
- Chặng 4–5: Viết bộ giả lập 50 máy ảo và bắt gói tin qua Wireshark.
- Chặng 6: Diễn tập kịch bản demo ngắn trước hội đồng chấm thi.

3.1 Chặng 1: CÔNG CỤ ĐÃ CÓ PR #9: Môi trường Stub độc lập (FakeAgent / FakeCentral)

3.1.1 Tại sao cần môi trường Stub?

Khi làm việc nhóm 4 người, nếu bạn phải chờ bạn A viết xong Agent hoặc bạn B viết xong Central mới được thử code của mình thì tiến độ sẽ bị tắc nghẽn hoàn toàn. Các thành phần giả lập (**Stubs**) giúp bạn tự do thử nghiệm độc lập:

- **FakeAgent:** Một chương trình nhỏ giả vờ là máy sinh viên, tự động kết nối và gửi các gói tin mẫu chuẩn.
- **FakeCentral:** Một server nhỏ mở cổng 5000, lắng nghe kết nối và phản hồi các gói tin mẫu.

3.1.2 Khi chạy, chuyện gì xảy ra?

Dùng mock harness hiện có theo README của tools/mock_harness. Bộ 16 fixture thuộc Protocol; không khẳng định FakeAgent đã chạy mọi fixture nếu chưa có ca tương ứng.

3.1.3 Khi nào coi là xong? (PASS Criteria)

- Gửi và nhận chính xác 16 fixture mẫu mà không bị lỗi giải mã.
- Toàn bộ khung tin trao đổi đều có 4 byte Big-Endian đúng quy định.

3.2 Chặng 2: THAM KHẢO TÙY CHỌN: Hàng đợi thu gom an toàn (Bounded Channel)

3.2.1 Tại sao cần phần này?

Trong thiết kế mở rộng tùy chọn, Collector xử lý thông điệp ứng dụng có thể cần hàng đợi khi tốc độ tiếp nhận lớn hơn tốc độ gửi tới Central. Hàng đợi không giới hạn có nguy cơ làm tăng bộ nhớ. Gateway L4 chỉ chuyển tiếp luồng byte theo kết nối, không gom hay đóng gói lại các thông điệp ứng dụng.

3.2.2 Khi chạy, chuyện gì xảy ra?

Hàng đợi có giới hạn để bộ nhớ không tăng vô hạn. Khi hàng đợi đầy, producer phải chờ để hệ thống tự giảm tốc độ tiếp nhận. Cơ chế bảo vệ này được gọi là **điều tiết áp lực ngược (Backpressure)**.

Ví dụ học tập dưới đây dành cho Collector, chưa được triển khai. Hàng đợi minh họa có 100 phần tử (`BoundedChannelOptions(100)`) với chính sách chờ (`BoundedChannelFullMode.Wait`):

- Duy nhất một luồng nền lấy từng gói tin ra để gửi tuần tự về Central Server.
- Khi đủ 100 phần tử, tác vụ thêm dữ liệu chờ bất đồng bộ cho tới khi có chỗ. Cần giới hạn cả kích thước phần tử và số tác vụ đang chờ; số phần tử riêng lẻ không bảo đảm toàn bộ bộ nhớ ổn định.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Hạn mức 100 chỉ giới hạn số phần tử đang nằm trong hàng đợi minh họa.
- Chính sách `FullMode.Wait` làm tác vụ ghi chờ; còn phải kiểm tra kích thước dữ liệu, hủy tác vụ và giới hạn tài nguyên.

```
// Pseudocode minh hoa: Hang doi thu gom du lieu Bounded Channel
public class TelemetryCollector
{
    private readonly Channel<byte[]> _queue = Channel.CreateBounded<byte[]>(
        new BoundedChannelOptions(100) { FullMode = BoundedChannelFullMode.Wait
            });

    public async ValueTask EnqueueAsync(byte[] frame, CancellationToken ct)
        => await _queue.Writer.WriteAsync(frame, ct);

    public async Task ForwardAsync(Stream stream, CancellationToken ct)
    {
        await foreach (var frame in _queue.Reader.ReadAllAsync(ct))
            await FrameCodec.WriteFrameAsync(stream, frame, ct);
    }
}
```

Listing 1: Pseudocode minh họa: Hàng đợi thu gom Bounded Channel và Backpressure

3.2.3 Khi nào coi là xong? (PASS Criteria)

- Hàng đợi giữ tối đa 100 phần tử, producer tự động tạm dừng đồng bộ nhịp độ khi đầy.
- Đo bộ nhớ và tác vụ chờ trong các ca thử dự kiến; chưa có kết quả thực nghiệm cho ví dụ này.

3.3 Chặng 3: THAM KHẢO TÙY CHỌN: Chuyển tiếp hai chiều (Duplex Relay) & Tự phục hồi (Failover)

3.3.1 Tại sao cần chuyển tiếp hai chiều?

Dữ liệu trong phòng máy không chỉ đi một chiều từ máy trạm lên máy chủ. Khi máy chủ Central muốn phản hồi gói tin xác nhận (`HEARTBEAT_ACK`) hoặc gửi lệnh điều khiển (ví dụ ACK hoặc phản hồi phiên trong thiết kế), làm sao gửi đúng đến chiếc máy đích nếu mọi máy trạm đều kết nối tập trung qua Gateway?

Gateway L4 cần giữ cặp kết nối Agent–Central để chuyển tiếp byte theo hai chiều. Collector L7 mới cần ánh xạ phiên để định tuyến thông điệp ứng dụng. Hai cách này là thiết kế tùy chọn khác nhau.

3.3.2 Khi chạy, chuyện gì xảy ra?

Ví dụ Collector có thể dùng bảng tra cứu phiên an toàn khi truy cập đồng thời (`Concurrent Dictionary`). Việc thêm, xóa và đóng kết nối vẫn cần quản lý vòng đời rõ ràng. Khi nhận bản tin điều khiển từ Central gửi về, Collector L7 có thể đọc phong bì để route phản hồi theo phiên; Gateway L4 chỉ relay byte, không đọc JSON để route từng thông điệp. Đây đều là thiết kế tùy chọn chưa triển khai.

3.3.3 Kịch bản tự phục hồi khi đứt mạng (Failover)

Đối với Collector, nếu kết nối tới Central bị gián đoạn, kế hoạch phục hồi cần xác định rõ:

1. Giới hạn bộ đệm và cách thông báo lỗi cho Agent; không xác nhận đã lưu khi Central chưa ghi SQL thành công.
2. Thử kết nối lại với thời gian chờ tăng dần, có giới hạn và hỗ trợ hủy tác vụ. Đây là kế hoạch, chưa có mã hoặc kiểm thử chứng minh.
3. Quyền gửi lại và chống trùng theo hợp đồng hiện hành; không tự tạo hàng đợi bền vững hoặc cam kết không mất dữ liệu. Gateway L4 không tự chuyển thông điệp giữa các phiên TCP mới.

3.4 Công cụ hỗ trợ: Bộ giả lập phòng máy (Simulator)

Tham khảo tùy chọn — Mã tác vụ: LG-D-006

3.4.1 Tại sao cần phần này?

Bộ giả lập `LabGuard.Simulator` hiện chỉ có placeholder. Theo kế hoạch tùy chọn, công cụ có thể mô phỏng 20–50 máy trạm để kiểm tra hệ thống; tải trên một máy không thay thế thử nghiệm LAN thực tế và chưa có kết quả để công bố.

3.4.2 Khi chạy, chuyện gì xảy ra?

Simulator khởi chạy 30–50 tác vụ `Task.Run` song song. Mỗi tác vụ đóng vai một máy sinh viên gửi nhịp tim mỗi 5 giây. Menu bàn phím cho phép bạn chủ động tạo sự cố:

- Phím [1] dự kiến: gửi ba mẫu CPU 95% liên tiếp cho máy ảo 05 để kiểm tra quy tắc HIGH_CPU khi bộ xử lý đã được hiện thực.
- Phím [2] dự kiến: gửi ba mẫu CPU 20% liên tiếp để kiểm tra điều kiện RECOVERED; giữ nguyên quy trình xử lý để người có quyền thao tác.
- Phím [3]: Rút mạng máy ảo 12 → Sau hơn 20 giây (> 20s), Central báo `OFFLINE`.
- Phím [4]: Cắm lại mạng cho máy ảo 12 → Kết nối lại bình thường.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- Mỗi máy ảo chạy độc lập trong vòng lặp `PeriodicTimer(5s)`.
- Biến cờ `IsNetworkCut` giúp mô phỏng tình huống rút dây mạng nhanh chóng.

```
// Pseudocode minh hoa: May ao VirtualAgent gui nhip tim va so lieu
while (await timer.WaitForNextTickAsync(ct))
{
    if (IsNetworkCut) continue; // Gia lap rut day mang
    await SendHeartbeatAsync(stream, ct);
    await SendMetricBatchAsync(stream, SimulatedCpu, ct);
}
```

Listing 2: Pseudocode minh họa: Vòng lặp mô phỏng VirtualAgent

3.4.3 Khi nào coi là xong? (PASS Criteria)

- Tạo 30–50 máy ảo đồng thời mà không làm đơ laptop.
- Menu phím bấm bơm lỗi phản hồi tức thời trên màn hình Admin.

3.5 Công cụ hỗ trợ: Bắt và phân tích gói tin bằng Wireshark

Chặng 5 — Mã tác vụ nội bộ: LG-D-005

3.5.1 Tại sao cần phần này?

Wireshark hỗ trợ kiểm chứng dữ liệu thực tế trên đường truyền, giúp bạn quan sát trực tiếp luồng byte nhị phân:

1. Mở Wireshark, chọn card mạng `Adapter for loopback` (nếu chạy nội bộ).
2. Gõ bộ lọc hiển thị: `tcp.port == 5000`.
3. Bấm vào một gói tin và chỉ cho thầy cô thấy:
   - Bắt tay 3 bước TCP (`[SYN]`, `[SYN, ACK]`, `[ACK]`).
   - 4 byte Big-Endian ở đầu khung tin (ví dụ: `00 00 00 9C` là 156 bytes).
   - Dữ liệu JSON của `WireEnvelope` với đúng 6 trường quy chuẩn.

3.5.2 Bảng soi từng byte khung tin trên Wireshark

Vị trí Byte	Giá trị Hex / Ký tự	Giải thích kỹ thuật
`0x0000--0x0003`	`00 00 00 9C`	4 byte độ dài Big-Endian = 156 byte payload theo sau.
`0x0004--0x0011`	`{"version":1,`	Phiên bản giao thức `version = 1`.
`0x0012--0x002B`	`"messageType":"HEARTBEAT",`	Tên thông điệp chuẩn trong `MessageTypes.cs`.
`0x002C--0x0058`	`"requestId":"d2a7...-90ef",`	Mã UUID v4 duy nhất 36 ký tự.
`0x0059--0x0078`	`"deviceId":"AGENT-LAB01-05",`	Tên thiết bị gửi bản tin.
`0x0079--0x00A8`	`"sentAtUtc":"2026-03-29...",`	Dấu thời gian UTC chuẩn có đuôi `Z`.
`0x00A9--0x009F`	`"payload":{...}}`	Nội dung nghiệp vụ chi tiết của thông điệp.

3.6 Kịch bản kiểm thử dự kiến cho nhiệm vụ chính của Ngọc

Đây là **kế hoạch kiểm thử**, không phải kết quả trình diễn đã đạt. Các bước dưới tập trung công cụ kiểm thử TCP và chức năng hỗ trợ xử lý sự cố trong Admin, không yêu cầu triển khai Gateway/Collector. Chỉ thực hiện từng bước khi có đủ mã, nguồn dữ liệu và điều kiện tích hợp tương ứng. Nếu chưa sẵn sàng, chỉ giải thích thiết kế hoặc dùng dữ liệu giả định có nhãn minh họa; không mô tả như phần mềm đã hoạt động.

Table 1: Các bước kiểm thử dự kiến và điều kiện thực hiện

Nội dung	Điều kiện thực hiện	Dự kiến kiểm tra và ghi nhận
Kết nối, gửi và nhận TCP	Có chương trình gửi/nhận chạy được, địa chỉ và cổng được phép; có thể dùng TcpEcho hoặc bộ mô phỏng độc lập đã có.	Gửi dữ liệu xác định trước, đối chiếu byte nhận và nhật ký kết nối/đóng kết nối. Ghi rõ chương trình và nguồn dữ liệu đã dùng.
Chuyển tiếp TCP	Chỉ chạy khi phần chuyển tiếp D-003 đã được viết và kiểm tra; không coi Gateway tùy chọn là điều kiện bắt buộc.	Đối chiếu dữ liệu ở hai đầu và cả hai chiều; kiểm tra đọc từng phần, ngắt kết nối và giải phóng tài nguyên. Chưa có mã thì ghi chưa thực hiện.
Dữ liệu sự cố có nguồn	Central và Admin đã hiện thực, tích hợp và kiểm chứng luồng dữ liệu/quyền; có dữ liệu sự cố được phép qua Central.	Đối chiếu thiết bị/sự cố đang chọn với dữ liệu Central. Hiện rõ nguồn, giá trị có thật và phần chưa được cung cấp; không tạo thời điểm hoặc lịch sử giả.
Bằng chứng và gợi ý	Có logic giải thích, bộ chuyển đổi dữ liệu và vùng hỗ trợ trong Admin đã được tích hợp; có ca dữ liệu đủ, thiếu và cũ được kiểm soát.	Phân biệt bằng chứng hiện có/thiếu/cũ; trình bày nhận định có giới hạn và đề xuất bước kiểm tra. Người có quyền quyết định thao tác; không tự kết luận nguyên nhân hoặc sửa máy.

Mở rộng tùy chọn: chỉ xem xét thử nhiều máy giả lập sau khi Simulator và các phần thu/lưu số liệu, nhịp tim, phát hiện sự cố, liên kết dữ liệu Admin đã được hiện thực, tích hợp và kiểm chứng. Hiện chưa cam kết Dashboard hiển thị một số lượng máy cụ thể, cảnh báo CPU hay OFFLINE. Kết quả chỉ được ghi sau khi thực chạy; không suy từ kịch bản dự kiến.

SAU PHẦN NÀY, BẠN CHỈ CẦN NHỚ:

- Mỗi kiểm tra TCP cần đối chiếu dữ liệu gửi/nhận và ghi đúng phần đã chạy; mô phỏng riêng không chứng minh toàn bộ LabGuard hoạt động.
- Chức năng hỗ trợ xử lý sự cố dùng dữ liệu có thẩm quyền qua Central, hiện giới hạn bằng chứng và gợi ý bước kiểm tra trong Admin.

4. Năm bẫy lỗi lập trình Gateway & Kiểm thử mạng

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 1: Tưởng lầm một lệnh Write trên TCP luôn tạo ra một gói tin mạng
TCP là dòng byte, một lần Read không nhất thiết là một message. Bên nhận ứng dụng hoặc Collector L7 cần đọc đúng frame; Gateway L4 có thể relay byte nguyên thứ tự mà không phân tích JSON.
Cách làm đúng: Phân biệt relay L4 với xử lý message L7. Không coi partial read là frame hoàn chỉnh; cũng không khẳng định relay byte đúng thứ tự tự làm hỏng framing.

ĐOẠN NÀY CẦN NHÌN ĐIỀU GÌ?

- L4 giữ byte và thứ tự, không đổi framing hoặc route theo JSON.
- L7 đọc đủ frame bằng FrameCodec khi cần xử lý từng message.

```
// L4: hai chieu rieng, giu thu tu byte; can shutdown/cancel phu hop.
await Task.WhenAll(
    agentStream.CopyToAsync(centralStream, ct),
    centralStream.CopyToAsync(agentStream, ct));

// L7 (vi du doc mot frame): khong coi mot Read la mot message.
byte[]? payload = await FrameCodec.ReadFrameAsync(agentStream, ct);
if (payload != null)
    await centralStream.WriteAsync(FrameCodec.Encode(payload), ct);
```

Listing 3: Pseudocode tham khảo: relay L4 và xử lý frame L7 là hai thiết kế khác nhau

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 2: Nghẽn đầu hàng (Head-of-Line Blocking) do dùng hàng đợi không giới hạn
Nếu sử dụng `Queue<byte[]>` không giới hạn dung lượng, khi mạng chậm gói tin sẽ dồn ứ vô tận trong RAM dẫn đến tràn bộ nhớ (*OutOfMemoryException*).
Cách làm đúng: Luôn sử dụng `Channel.CreateBounded` với giới hạn dung lượng an toàn để tránh tăng RAM vô hạn.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 3: Tranh chấp truy cập bảng định tuyến phiên
Khi 50 máy con kết nối và ngắt kết nối đồng thời, việc tra cứu socket đích sẽ gây lỗi đa luồng nếu dùng `Dictionary` thông thường.
Cách làm đúng: Luôn dùng `ConcurrentDictionary<string, TcpClient>`.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 4: Tạo quá nhiều Thread độc lập trong bộ giả lập Simulator
Nếu tạo 50 máy ảo bằng cách gõ 50 lần `new Thread()`, máy tính sẽ bị ngốn hàng chục MB bộ nhớ và làm CPU giật lag do chuyển ngữ cảnh liên tục.
Cách làm đúng: Tận dụng `Task.Run` kết hợp `async/await` và `PeriodicTimer`.

CẢNH BÁO BẪY LỖI & THỰC HÀNH CẦN TRÁNH

Bẫy 5: Quên giải phóng kết nối khi máy trạm ngắt đột ngột
Khi máy trạm bị tắt nguồn hoặc rút cáp, nếu vòng lặp không bắt lỗi đóng luồng thì socket sẽ bị treo vĩnh viễn, làm rò rỉ tài nguyên mạng của hệ thống.
Cách làm đúng: Luôn đặt luồng socket trong khối lệnh `using`.

5. Câu hỏi tham khảo TCP và mở rộng tùy chọn (20 câu)

Các câu về Gateway/Collector/Simulator bên dưới là kiến thức tham khảo cho mở rộng tùy chọn, không chứng minh đã có mã hoặc kết quả kiểm thử. Nhiệm vụ chính của Ngọc vẫn là công cụ kiểm thử TCP và hỗ trợ phân tích, xử lý sự cố tích hợp trong LabGuard.

1. **Tại sao phải dùng Wireshark để soi gói tin thay vì chỉ đọc log trên console?**
   Trả lời: Wireshark hỗ trợ kiểm chứng dữ liệu thực tế trên đường truyền ở tầng truyền vận (TCP/IP), chứng minh hệ thống tuân thủ chặt chẽ tiêu chuẩn đóng khung 4 byte Big-Endian và mô hình kết nối tin cậy.

2. **Bộ lọc nào trên Wireshark cho phép cô lập chính xác dữ liệu của LabGuard?**
   Trả lời: Dùng biểu thức lọc hiển thị `tcp.port == 5000` để thu thập toàn bộ các gói tin TCP trao đổi giữa Agent, Gateway và Central Server.

3. **Tại sao đồ án cần xây dựng các bản Stub (FakeAgent, FakeCentral)?**
   Trả lời: Nhằm phá vỡ sự phụ thuộc chéo về tiến độ giữa các thành viên, cho phép kiểm thử độc lập chức năng chuyển tiếp của Gateway ngay từ những ngày đầu.

4. **Bộ giả lập (Simulator) có vai trò gì trong buổi nghiệm thu đồ án?**
   Trả lời: Giúp tái hiện môi trường thực tế với 20–50 máy trạm hoạt động đồng thời, đóng vai trò công cụ hỗ trợ kiểm chứng năng lực chịu tải và thuật toán xử lý sự cố đa luồng của máy chủ.

5. **Tại sao nhịp tim gửi 5 giây một lần mà lại chờ hơn 20 giây mới báo Offline?**
   Trả lời: Nhằm tránh báo động giả do trễ mạng hoặc suy giảm kết nối nhất thời. Máy chủ quy định ngưỡng quá hạn gấp 4 lần chu kỳ gửi ($4 \times 5\text{s} = 20\text{s}$).

6. **Làm thế nào để Gateway tránh việc làm nghẽn mạng WAN?**
   Trả lời: Bằng kỹ thuật gom cụm (Batching) nhiều bản tin đo đạc nhỏ từ các Agent trong phòng máy vào hàng đợi Bounded Channel trước khi chuyển tiếp thành từng khối lớn về Central Server.

7. **Tại sao hàng đợi thu gom của Gateway phải là hàng đợi có giới hạn (Bounded)?**
   Trả lời: Để ngăn ngừa nguy cơ cạn kiệt bộ nhớ RAM khi đường truyền mạng WAN bị gián đoạn hoặc quá tải. Hàng đợi giới hạn 100 phần tử kích hoạt cơ chế điều tiết áp lực ngược (Backpressure) với chính sách `FullMode.Wait`, buộc bên gửi tạm dừng bất đồng bộ để hệ thống tự điều hòa nhịp độ an toàn.

8. **Cơ chế Exponential Backoff trong quá trình kết nối lại hoạt động ra sao?**
   Trả lời: Sau mỗi lần kết nối lại thất bại, thời gian chờ tăng gấp đôi (1s, 2s, 4s, 8s... trần 30s), tránh gây ra hiện tượng bão kết nối làm tê liệt máy chủ.

9. **Làm thế nào để Simulator tạo ra 50 máy ảo mà không làm treo ứng dụng?**
   Trả lời: Dùng `Task.Run` tận dụng ThreadPool và cơ chế I/O Completion Ports (IOCP) của hệ điều hành, cho phép tạo 50–100 máy ảo chạy trên chỉ vài luồng hệ thống mà không tốn RAM.

10. **Tiêu đề 4 byte Big-Endian hiển thị ra sao trong cửa sổ Byte Viewer của Wireshark?**
Trả lời: Hiển thị dưới dạng 4 byte hex liên tiếp ở đầu payload TCP, ví dụ: `00 00 00 A4` nghĩa là khung tin dài 164 byte.

11. **Tại sao phải chờ hơn 20 giây thì Central Server mới báo máy trạm bị ngắt mạng?**
Trả lời: Vì chu kỳ nhịp tim là 5 giây. Máy chủ cho phép trễ tối đa 4 chu kỳ liên tiếp (tổng cộng > 20 giây) để tránh báo động giả khi mạng phòng máy bị nghẽn tức thời.

12. **Kỹ thuật Bơm lỗi (Fault Injection) mang lại lợi ích gì cho đồ án?**
Trả lời: Cho phép chủ động kiểm chứng độ tin cậy và khả năng tự phục hồi của toàn bộ hệ thống ngay tại bàn thi mà không cần can thiệp vào thiết bị phần cứng thật.

13. **Làm thế nào để đo độ trễ mạng (RTT) của gói tin nhịp tim?**
Trả lời: Tính khoảng thời gian từ khi Client gửi bản tin `HEARTBEAT` đến khi nhận được bản tin `HEARTBEAT_ACK` tương ứng từ Central Server.

14. **Giới hạn khung tin 256 KiB có ảnh hưởng gì tới tầng Gateway không?**
Trả lời: Gateway L4 giữ stream byte, không giải mã JSON. Bên ứng dụng hoặc Collector L7 kiểm tra giới hạn qua FrameCodec. Cả hai là kiến thức mở rộng tùy chọn, chưa có nghiệm thu triển khai.

15. **Tại sao khi thử tải lại không chạy trên 50 máy tính thật?**
Trả lời: Vì sinh viên không thể có 50 máy tính thật để thử nghiệm hàng ngày. Bộ giả lập chứng minh năng lực thiết kế phần mềm có thể đáp ứng quy mô lớn ngay trên môi trường phát triển.

16. **Hiện tượng SYN Flood là gì và ảnh hưởng gì tới máy chủ?**
Trả lời: Kẻ tấn công gửi dồn dập các gói tin `SYN` giả mạo mà không gửi lại gói `ACK` thứ ba, làm cạn kiệt hàng đợi kết nối đang chờ (*Backlog Queue*) của máy chủ.

17. **Cơ chế Backlog trong lệnh Listen(int backlog) có ý nghĩa gì?**
Trả lời: Là độ dài tối đa của hàng đợi chứa các kết nối đã hoàn thành bắt tay TCP 3 bước nhưng ứng dụng chưa kịp gọi hàm `Accept`.

18. **Làm sao để đảm bảo các máy trạm ảo trong Simulator không gửi trùng mã RequestId?**
Trả lời: Mỗi thông điệp đều sinh mã `Guid.NewGuid().ToString("D")` độc lập, bảo đảm tính duy nhất tuyệt đối trên toàn hệ thống.

19. **Khi tắt đột ngột chương trình Simulator, chuyện gì xảy ra trên máy chủ?**
Trả lời: Hệ điều hành Windows sẽ tự động gửi gói tin cờ `RST` hoặc `FIN` tới máy chủ, máy chủ lập tức giải phóng toàn bộ tài nguyên socket của các máy ảo đó.

20. **Mục tiêu chính trong phần việc của Thành viên D là gì?**
Trả lời: Kiểm tra kết nối, gửi, nhận và chuyển tiếp TCP; phát triển chức năng hỗ trợ phân tích, xử lý sự cố trong Admin bằng dữ liệu qua Central. Phân biệt bằng chứng hiện có, thiếu và cũ để gợi ý bước kiểm tra; không tự kết luận nguyên nhân hoặc sửa máy. Đây là mục tiêu phát triển, không phải tuyên bố đã hoàn thành. Gateway/Collector chỉ là mở rộng tùy chọn.

— HẾT CẨM NANG HƯỚNG DẪN THÀNH VIÊN D —